

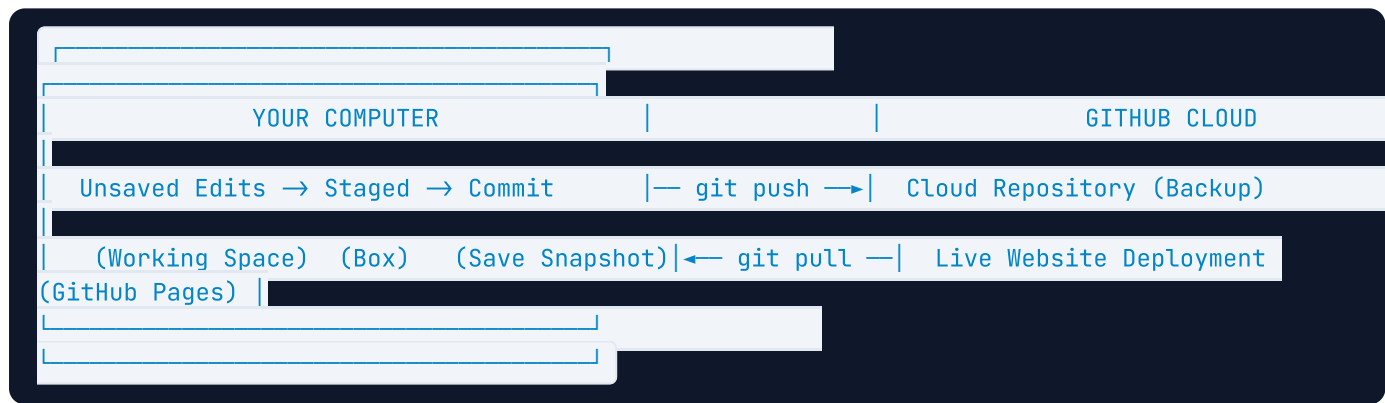
The Vibe Coder's Guide to Git & GitHub: Industry Workflows Made Simple

Welcome! If you love to "vibe code" — building features intuitively, experimenting rapidly, and seeing ideas come to life — but feel confused by Git terminal commands, branches, pushes, pulls, and industry workflows, **this guide is for you.**

1. What is Git vs. GitHub? (The Video Game Analogy)

Think of coding like playing a massive open-world video game:

- **Git** is your **Local Save Engine**. It runs on your computer and creates "save checkpoints" of your code files so you can rewind time if something breaks.
- **GitHub** is your **Cloud Save & Multiplayer Server**. It backs up your save files on Google/Microsoft cloud servers, hosts your live website, and lets other developers collaborate with you.



2. Core Concepts Demystified (No Jargon!)

Term	What it actually means	Real-World Analogy
Repository (Repo)	The project folder containing all your code files and Git history.	The game save folder.
Working Directory	Files you are currently editing in VS Code / IDE before saving them to Git.	Unsaved game progress.
Staging Area (<code>git add</code>)	Selecting which modified files you want to include in your next save point.	Placing items into a shipping box before sealing it.
Commit (<code>git commit</code>)	Taking a permanent, named snapshot of your staged files at a specific moment.	Sealing the box with a date label (a Save Point).
Branch (<code>git branch</code>)	Creating an isolated parallel universe of your project to test new code safely.	Creating a duplicate save slot to try a risky boss fight.
Main Branch (<code>main</code>)	The primary, official version of your project that feeds your live website.	The official public release of your game.
Push (<code>git push</code>)	Uploading your local commits from your computer up to GitHub's cloud servers.	Syncing your local save file to Cloud Storage.
Pull (<code>git pull</code>)	Downloading the latest commits from GitHub down to your local computer.	Syncing cloud saves down to your laptop.
Merge (<code>git merge</code>)	Combining code from an experimental branch back into the <code>main</code> branch.	Merging your sandbox experiment back into the main game.
Pull Request (PR)	A proposal page on GitHub asking to merge your branch changes into <code>main</code> .	Submitting your mod to be approved into the official game.

3. The Professional 4-Step Industry Workflow

In software engineering, developers never code directly on the `main` branch. They follow a clean 4-step workflow:

Step 1: Create Branch → Step 2: Vibe Code & Commit → Step 3: Push Branch → Step 4: Merge to Main

(`git checkout -b`) (`git add . && git commit`) (`git push origin`) (Live Website Deploys!)

Step 1: Create a Sandbox Branch (Stay Safe!)

Before tweaking code or trying a new feature, create a new branch so your working `main` site stays untouched:

```
git checkout -b feature/audio-effects
```

Now you are in an isolated sandbox. Whatever you break here won't affect your live website!

Step 2: Vibe Code & Make Checkpoints

As you code and get things working, save your progress in small checkpoints:

```
# 1. Select all modified files
git add .

# 2. Lock in a save snapshot with a short description
git commit -m "feat: Add flip sound FX synthesizer"
```

Step 3: Push Your Branch to GitHub Cloud

Upload your branch to GitHub so your work is backed up online:

```
git push origin feature/audio-effects
```

Step 4: Merge into `main` to Release to the World!

Once your new feature works great and passes tests, combine it into `main`:

```
# 1. Switch back to main branch
git checkout main

# 2. Merge your feature branch into main
git merge feature/audio-effects

# 3. Push main to GitHub (Triggers live site deployment!)
git push origin main
```

4. Real-Life Scenarios for Vibe Coders

Scenario A: "I broke everything and want to undo my unsaved edits!"

If you edited code in VS Code, everything is broken, and you want to discard unsaved changes and reset back to your last working commit:

```
# Revert all modified files back to the last clean save point
git restore .
```

Scenario B: "I'm switching from my Laptop to my Desktop computer."

- On Laptop before leaving:

```
bash git add . git commit -m "wip: Work in progress on quiz modal"
git push origin main
```
- On Desktop when opening project:

```
bash git pull origin main
```

Scenario C: "I created a branch and want to delete it after merging."

Once your branch code is safely inside `main`, clean up your old branch:

```
# Delete branch locally
git branch -d feature/audio-effects

# Delete branch on GitHub cloud
git push origin --delete feature/audio-effects
```

5. Daily Git Cheat Sheet (The Only 10 Commands You Need)

Action	Command	What it does
Check Status	<code>git status</code>	Shows which files were modified, created, or deleted.
Stage Changes	<code>git add .</code>	Stages ALL modified files for the next commit.
Commit Save	<code>git commit -m "your message"</code>	Creates a permanent save snapshot with a title.
Upload Code	<code>git push origin main</code>	Uploads local <code>main</code> commits to GitHub cloud.
Download Code	<code>git pull origin main</code>	Downloads latest cloud updates to your computer.
Create Branch	<code>git checkout -b branch-name</code>	Creates and switches to a new experimental branch.
Switch Branch	<code>git checkout branch-name</code>	Switches your working files to an existing branch.
List Branches	<code>git branch</code>	Shows all branches (the active one has an <code>*</code>).
Merge Branch	<code>git merge branch-name</code>	Combines <code>branch-name</code> into your current active branch.
View History	<code>git log --oneline -n 5</code>	Displays your last 5 save commits in a clean list.

6. Pro-Tips for Vibe Coders

- 1. **Commit Often:** Don't wait 3 days to make one huge commit. Make small commits every time a sub-feature works (`"add sound button"`, `"fix dark mode contrast"`). Small commits make undoing mistakes easy!
- 2. **Write Clear Commit Messages:** Use prefixes like `feat:` (new feature), `fix:` (bug fix), or `docs:` (documentation updates).
- 3. **Check `git status` First:** Whenever you feel unsure of where you are, type `git status` . It tells you your active branch and modified files.

7. Deep-Dive: How Git Stores History (Commits & Diffs)

What is a Commit SHA Hash?

Every commit you create is assigned a unique 40-character fingerprint hash (e.g. `243a4391a8e2...`). You can view recent commits using `git log --oneline`:

```
243a439 (HEAD → main, origin/main) docs: Add Git & GitHub guide for vibe coders
2439c75 feat: Add 30 automated test cases, fix selective deck reset & hotkey edge cases
15b801a feat: Character Grid Progress Chart, visual Kanji emojis & dark mode UI
```

Line-by-Line Diffs (`git diff`)

Git doesn't save entire duplicate copies of your files on every save. Instead, it tracks exact line-by-line additions (`+`) and deletions (`-`):

```
- markBtn.textContent = "Mark Known";
+ markBtn.textContent = known.has(card.c) ? "✓ Known" : "Mark Known";
```

8. Solving Merge Conflicts Without Panicking

What is a Merge Conflict?

A **Merge Conflict** happens when two branches edit the exact same line of code in different ways. Git doesn't know which version you prefer, so it pauses and asks you to pick!

```
<<<<<< HEAD (Current Branch / main)
const themeColor = "#c0392b";
=====
const themeColor = "#1a5276";
>>>>>> feature/blue-theme (Incoming Branch)
```

3 Steps to Resolve Merge Conflicts Easily:

1. Open the conflicted file in **VS Code**. VS Code will highlight the conflicting lines with helpful buttons:
2. `Accept Current Change` (Keep `main` version)
3. `Accept Incoming Change` (Keep feature branch version)
4. `Accept Both Changes`
5. Click your preferred button to resolve the conflict.
6. Save the file and run: `bash git add . git commit -m "fix: Resolve merge conflict between main and feature branch" git push origin main`

9. Managing Secrets & Security (`.gitignore`)

Never Commit Secret API Keys or Passwords!

Once you push code to a public GitHub repository, automated bots scan GitHub 24/7 looking for leaked passwords, database keys, or private tokens.

How `.gitignore` Protects You:

The `.gitignore` file lists files and folders that Git should **completely ignore and never track**:

```
# Exclude local build caches
.firebase/
node_modules/

# Exclude private environment keys
.env
.env.local
secrets.json
```

10. Git Stash (The Temporary Pocket)

What is Git Stash?

Imagine you are halfway through writing new code on `main`, but suddenly you need to pull the latest production updates without committing your half-baked work.

`git stash` acts like a **temporary coat pocket**. It stashes your uncommitted edits away safely so your project returns to a clean state.

```
# 1. Put unsaved work into temporary stash pocket
git stash

# 2. Perform your pull or switch branches cleanly
git pull origin main

# 3. Take your unsaved work back out of your stash pocket
git stash pop
```

11. Undoing Mistakes: The Safety Net Guide

Scenario	Command to Run
Discard unsaved local edits	<code>`git restore .`</code>
Unstage files added by mistake	<code>`git restore --staged <filename>`</code>
Fix message of last unpushed commit	<code>`git commit --amend -m "new message"`</code>
Undo last commit (keep code edits)	<code>`git reset HEAD~1`</code>
Safely undo a published commit on main	<code>`git revert <commit-id>`</code>

12. Using VS Code Visual GUI Controls

If you prefer clicking buttons over typing terminal commands, **VS Code has built-in Git integration**:

1. Click the **Source Control Icon** on the left sidebar (or press `Ctrl + Shift + G`).
2. Modified files are listed under **Changes**. Click the `+` icon next to a file to stage it (`git add`).
3. Type your message in the top text box and click **Commit**.
4. Click **Sync Changes** to push and pull from GitHub automatically!